Questions of the day

- Is it possible to improve the greedy approach so that it always finds the right answer?
  - What's the cost?
- How can we apply graph algorithms to *any* problem?
- How can a computer scientist make $1,000,000 for writing a paper?

Backtracking and introduction to Complexity Theory

William Hendrix

Lecture 12

Outline

- Review
  - Topological sorting
  - Ford-Fulkerson
- Backtracking
- Problem complexity
- Classes P and NP
- Problem reductions
- Classes NP-Hard and NP-Complete
- NPC example problems

Review: topological sorting and FF

- **Topological sorting**
  - Order vertices of DAG to meet all prereqs
  - Perform DFS and add vertices to stack as *explored*
  - Reverse stack to sort
  - *Complexity:* same as DFS
    - $\Theta(n + m)$
- **Ford-Fulkerson**
  - Finds max-flow and min-cut in source-sink network
  - Set all flows to 0
  - Find all vertices reachable with positive tolerance
  - If you reach sink, this yields augmenting path
    - Increase flow by *tolerance*
      - Available capacity on forwards edges
      - Flow on backwards edges
    - Repeat
  - Otherwise, return cut with S = {positive tolerance vertices}

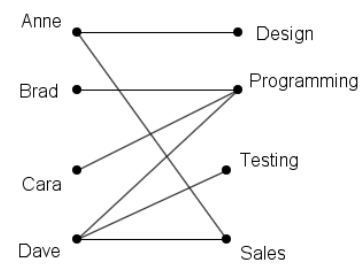
Ford-Fulkerson complexity

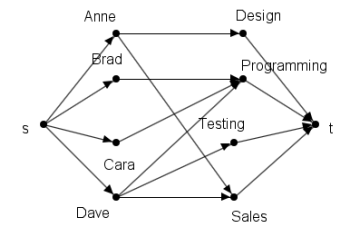
1. Start at source
2. Mark all vertices with positive tolerance
3. Choose an unvisited marked vertex
4. Repeat steps 2 & 3 until you reach the sink or run out of vertices
5. Start over if you reached the sink
6. Otherwise, return set of reachable vertices

- Every vertex other than source needs at least one incoming edge
  - $n = O(m)$
- Finding an augmenting path: $O(n + m) = O(m)$
- How many times can we start over?
  - Each augmenting path increases flow
  - $O(f)$ iterations
- Better: search for augmenting flow with BFS
  - Always finds shortest augmenting path
  - Each augmenting flow will "max out" one edge
    - Future flows can reduce this edge, but only by finding a longer AP
  - $O(nm)$ iterations
  - $O(nm^2)$ total
- Can be improved to $O(n^3)$ using push-relabel algorithms

Max-flow, min-cut applications

- **Edge connectivity**
  - Given a graph, find the fewest number of edges whose removal disconnects the graph
  - Measure of network stability
  - Algorithm
    - Choose one vertex as source
    - Calculate min cut with every other vertex as sink
    - Return the smallest min cut
  - Complexity: $\Theta(n^4)$ or $\Theta(n^2m^2)$

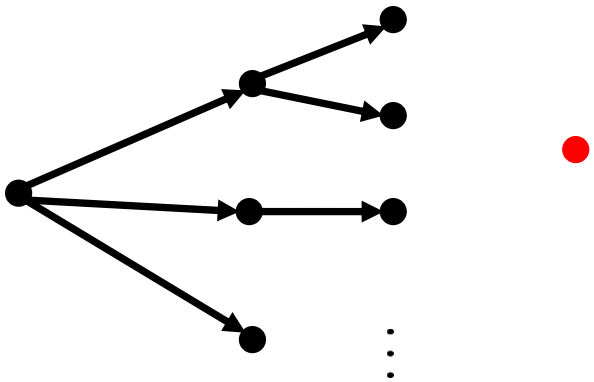




- **Maximum matching**
  - Given a graph (usually bipartite), find maximum number of edges that share no endpoints
  - E.g., matching workers to jobs, events to timeslots, etc.
  - Algorithm
    - Add source and sink nodes, each connected to vertices on one side with cap 1
    - Set all capacities to 1
    - Max matching = edges with flow

Backtracking

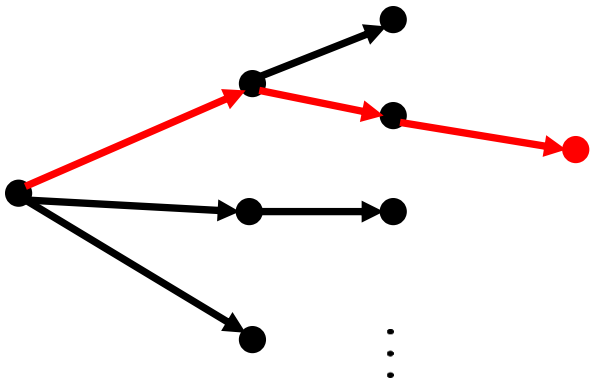
- Algorithm strategy based on graph traversal
  - Can be exact or heuristic



- **Main idea**
  - Model problem as sequence of decisions
  - Set of states and decisions defines a graph
  - Traverse with DFS

Backtracking

- Algorithm strategy based on graph traversal
  - Can be exact or heuristic



- **Main idea**
  - Model problem as sequence of decisions
  - Set of states and decisions defines a graph
  - Traverse with DFS

- **Complexity:** $\Theta(n+m) = \Theta(n)$ if tree

- How big is this graph?
  - Contains all possible solutions
  - Usually exponential

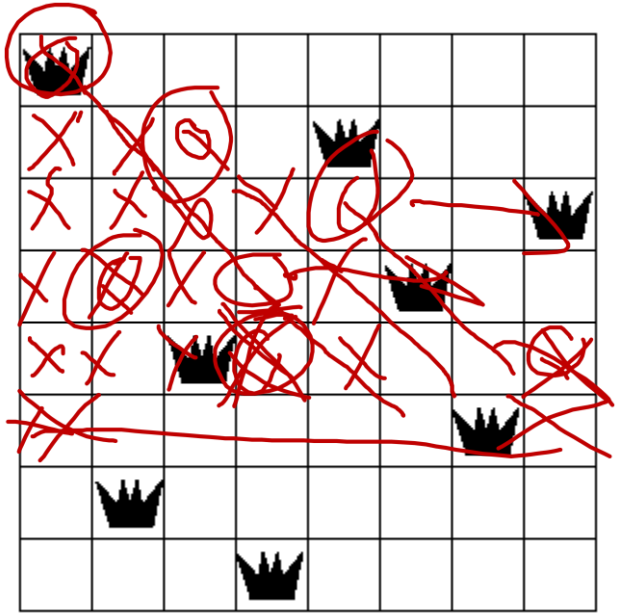
Improving naïve backtracking

- Don't construct entire state graph!
  - Keep track of location and candidates
- Don't revisit same state
- Don't explore states that can't lead to the solution
  - *Pruning* or *branch and bound*

- How you explore search space matters
  - Best order depends on problem
- Explore promising candidates first
  - Stop when you know you've found solution
  - E.g., A*
- Explore candidates that eliminate the most possibilities first
- Heuristic backtracking
  - Only explore most likely candidates
  - E.g., beam search

Backtracking example

- Eight queens puzzle
  - Can you place 8 queens on an 8x8 chessboard that don't attack one another?



- **Observation**
  - One queen in every row and column
- **Strategy:** row-by-row
  - Choose column numbers for queens
  - Keep track of current assignments
  - "Disable" columns as you go
  - Check backwards diagonals to eliminate possibilities
  - Backtrack if all columns are eliminated
  - Stop when all 8 are placed

Eight queens algorithm

```
Algorithm: EightQueens
queen = Array(8)
col = BoolArray(8)
Initialize col to true
Call EQBT(1)
Output solution
```

```
Algorithm: EQBT(rownum)
for c = 1 to 8 do
    if col[c] is true then
        if any previous queen is diagonal from
          (rownum, c) then
            Advance c to next column
        else
            queen[rownum] = c
            col[c] = false
            Call EQBT(rownum + 1)
            if solution not found then
                col[c] = true
            else
                return solution
            end
        end
    end
end
return
```

Eight queens analysis

- Don't construct entire state graph
  - *queen* stores state
- Don't revisit same state
  - For loop on columns checks each column once
- Don't explore states that can't lead to the solution
  - *col* and diagonal check

- Explore promising candidates first
  - No clear "promising" column
- Explore candidates that eliminate the most possibilities first
  - Center squares first?
- Heuristic backtracking
  - Not used

Backtracking exercise

- Sudoku
  - **Input:** partially filled 9x9 grid of digits
  - **Output:** fully filled 9x9 grid such that no digit is repeated in each row, column, or 3x3 block

for $u \in N(v)$

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

1. How do you represent state?
2. What are the "decisions" you can make?
3. How do you avoid repeating yourself?
4. When do you backtrack?
5. What moves should you explore first?

1. Partial 9x9 grid
2. add # to □
3. Pick □, iterate
4. □ w/ no legal digits

5. □ w/ few options / frequent digits

Backtracking sample solution

- Sudoku
  - **Input:** partially filled 9x9 grid of digits
  - **Output:** fully filled 9x9 grid such that no digit is repeated in each row, column, or 3x3 block

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

1. Array with currently filled-in digits
2. What square to fill in and what digit
3. Pick a square, then try each possible digit once
4. When the problem constraints are violated
   - OR: when a square has zero possibilities
5. Squares with the fewest number of options
   - OR: square with most frequent digit on board as possibility

Backtracking summary

Pros

- Can solve difficult problems in feasible amount of time
- Flexible
- Can be modified to create heuristic

Cons

- Worst case behavior often exponential
- Average behavior difficult to analyze
- Not as straightforward as greedy approach

Complexity theory motivation

- Complexity classes
  - $\Theta(n)$, $\Theta(n \lg n)$, etc.

Infeasible

n=	lg(n)	n	n lg(n)	n^2	n^3	2^n	$n!$
10	3 ns	10 ns	33 ns	100 ns	1 µs	1 µs	3.6 ms
20	4 ns	20 ns	86 ns	400 ns	8 µs	1 ms	77 yrs
30	5 ns	30 ns	147 ns	900 ns	27 µs	1 s	
40	5 ns	40 ns	213 ns	1.6 µs	64 µs	18.3 min	
50	6 ns	50 ns	282 ns	2.5 µs	125 µs	13 days	
100	7 ns	100 ns	664 ns	10 µs	1 ms		
1,000	10 ns	1 µs	9.97 µs	1 ms	1 s		
1,000,000	20 ns	1 ms	19.9 ms	16.7 min	31.7 yrs		
1,000,000,000	30 ns	1 s	29.9 s	31.7 yrs			

"Fails" at:	Never!	trillions	millions	10k	40ish	16ish

- Important distinction: polynomial vs. exponential

Complexity theory (not on Final!)

- Branch of computer science concerned with *problem complexity*
  - Complexity of fastest *possible* algorithm for a problem
  - **Example:** comparison-based sorting: $\Theta(n \lg n)$
- Identifying problem complexity is very difficult
  - Sometimes we establish bounds instead
    - E.g., $O(n^3)$ or $\Omega(\lg n)$
- Main goal is to distinguish *feasible* problems from *infeasible*
  - Polynomial vs. exponential

- **Class *P***
  - Problems that can be solved in polynomial time*
  - To prove *p* is in *P*: find a poly-time algorithm
  - To prove *p* is not in *P*: show there is no poly-time algorithm

Much harder!

Decision problems

- Formally, *P* and other classes defined in terms of *decision problems*
- Problem with true/false answer
  - E.g., is there a path of length *k* or less from *u* to *v* in *G*?
- Relationship to theory of computation
  - Set of true inputs forms a *language*
  - All problems are essentially languages
  - AT tells us minimal model of computation necessary
- Relationship to optimization problems
  - Decision problems form the "core" hardness of optimization
  - **Example:** solving shortest path

```
Algorithm: Short-Path(u, v)
for k = 0 to n − 1 do
    if Short-Path(u, v, k) then
        return k
    end
end
error no (u, v)-path in G
```

Decision problems

- Formally, *P* and other classes defined in terms of *decision problems*
- Problem with true/false answer
  - E.g., is there a path of length *k* or less from *u* to *v* in *G*?
- Relationship to theory of computation
  - Set of true inputs forms a *language*
  - All problems are essentially languages
  - AT tells us minimal model of computation necessary
- Relationship to optimization problems
  - Decision problems form the "core" hardness of optimization
  - **Example:** solving shortest path

```
Algorithm: Short-Path(u, v)
for k = 0 to n − 1 do
    if Short-Path(u, v, k) then
        return k
    end
end
error no (u, v)-path in G
```

<u>Complexity</u>

Θ(n) times `Short-Path`

- Could be Θ(lg *n*) factor with one-sided binary search
- Doesn't change *"P-ality"*

Problem reduction

- Primary technique to prove problem complexity
  - Algorithmic strategy

- **Main idea**
  - Suppose problem A is similar to another problem, B
  - Transform input *a* into corresponding problem *b*
  - Solve *b*
  - Transform solution to *b* into solution for *a*

- Complexity to solve *a*:
  - B complexity + transform complexity
  - If B and T are poly-time, A is poly-time
  - T is poly-time: poly-time problem reduction
    - Poly-time reductions can show class P

Reduction example

- Decreasing-Sort
  - **Input:** *data:* array of n integers
  - **Output:** permutation of data such that $data[1] \geq data[2] \geq \ldots \geq data[n]$
- **Related problem:** ordinary sort
- **Example reduction**
  - Negate elements of *data*
  - Sort transformed *data*
  - Negate elements of *data* again
  - Solves Decreasing-Sort using Sort
    - Reduction from DS to Sort
- **Complexity**
  - $O(n) + O(n \lg n) + O(n) = O(n \lg n)$
  - Problem complexity of DS must be $O(n \lg n)$

Another reduction

- Median
  - **Input:** *data:* array of n integers
  - **Output:** median of *data*
- **Related problem:** sorting
- **Example reduction:**
  - Sort *data*
  - Return $data[n/2]$
- **Complexity:**
  - $\Theta(n \lg n)$
  - Median complexity must be $O(n \lg n)$
  - Not optimal; there is $O(n)$ algorithm

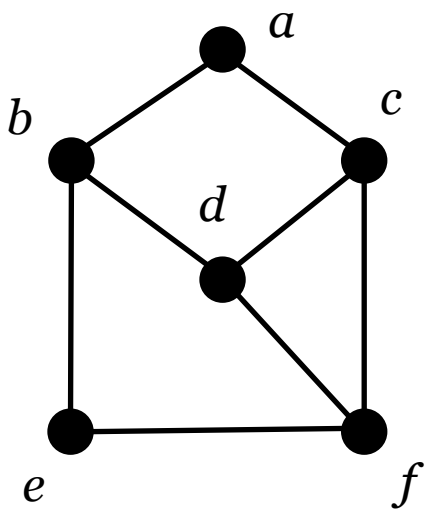
Class NP

- Nondeterministic polynomial time
  - Not "non-polynomial"
- Could be solved in polynomial time by a nondeterministic algorithm
  - Nondeterministic algorithm/Turing machine:
    - When given a choice, try them all simultaneously and pick the best
    - Or: "perfectly lucky"
- Equivalently, problems that can be *verified* in poly-time

- **Nondeterministic time example:** sorting
  - Comparison-based sorting takes $\Theta(n \lg n)$, in general
  - Verifying that an array is sorted takes $\Theta(n)$
    - *Nondeterministic:* guess the min value and swap to next position

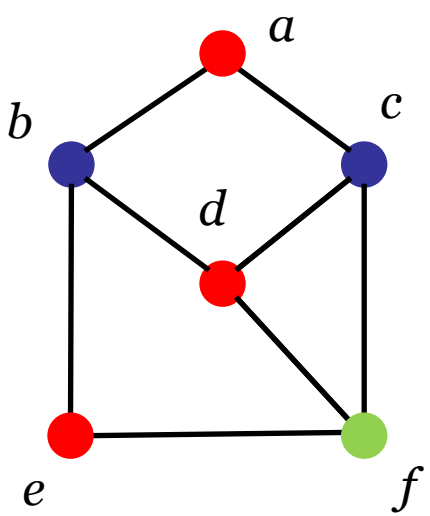
Proving NP

- More useful definition based on verification
- **To prove:** find a poly-time verification algorithm
  - Input will be same as original, *plus a proposed solution*
  - Algorithm should verify solution solves problem
- **Example:** graph coloring
- Does graph *G* have a valid coloring with *k* colors?
  - Assigns every vertex a color
  - Adjacent vertices must be different colors



Proving NP

- More useful definition based on verification
- **To prove:** find a poly-time verification algorithm
  - Input will be same as original, *plus a proposed solution*
  - Algorithm should verify solution solves problem
- **Example:** graph coloring
- Does graph *G* have a valid coloring with *k* colors?
  - Assigns every vertex a color
  - Adjacent vertices must be different colors



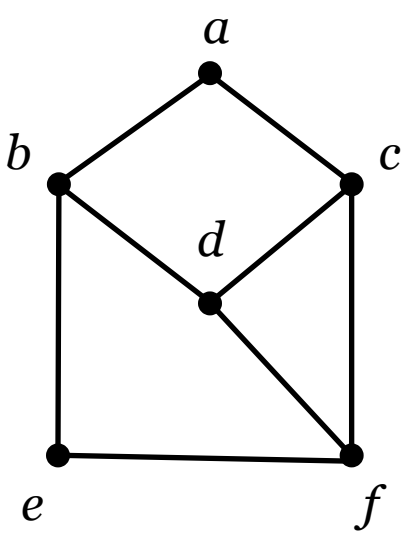
Given assignment of colors
For each vertex, check if any neighbors match
If so, not a valid coloring
Count number of different colors (set)
Return whether coloring uses *k* colors

Complexity: $O(m + n\lg n)$

- Adj matrix for graph, BBST for counting

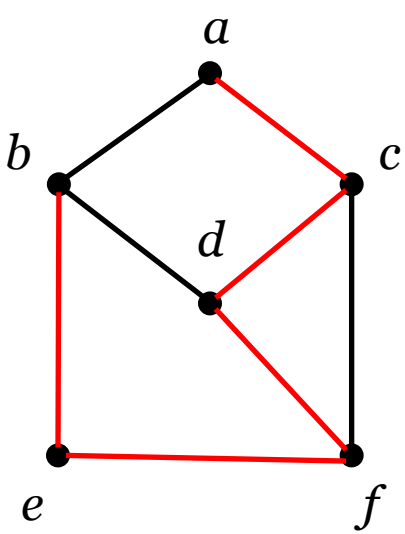
NP problem example

- Given a graph G with vertices u and v, find the longest path from u to v in G.
- Or: is there a path of length k or more between u and v?



NP problem example

- Given a graph G with vertices u and v, find the longest path from u to v in G.
- Or: is there a path of length k or more between u and v?

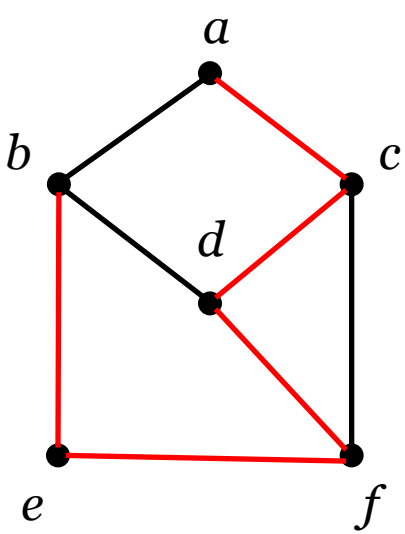


- No obvious poly-time algorithm
- Exponential time DP and brute force algorithms

NP problem example

- Given a graph G with vertices u and v, find the longest path from u to v in G.
- Or: is there a path of length k or more between u and v?

Solution verification

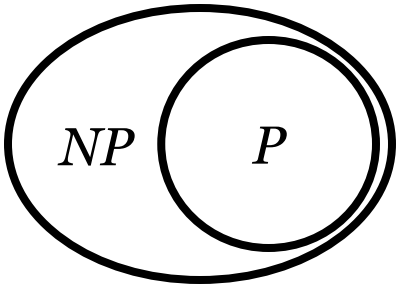


Given path $P : u, p_1, p_2, \ldots, p_{k-1}, v$
Check $(u, p_1), (p_1, p_2), \ldots, (p_{k-1}, v)$ are all edges
Check no vertices are repeated
Check $|P| \geq k$
Conclude `LongPath`(u, v) must be at least k

Complexity: $O(k)$ with adj. matrix

P vs. NP

- *P:* decision problems that can be solved in poly-time
- *NP:* decision problems that can be verified in poly-time
  - Or solved in *nondeterministic poly-time*
- Every problem in *P* is in *NP*
  - Solve the problem and check if decision is same
  - $P \subseteq NP$

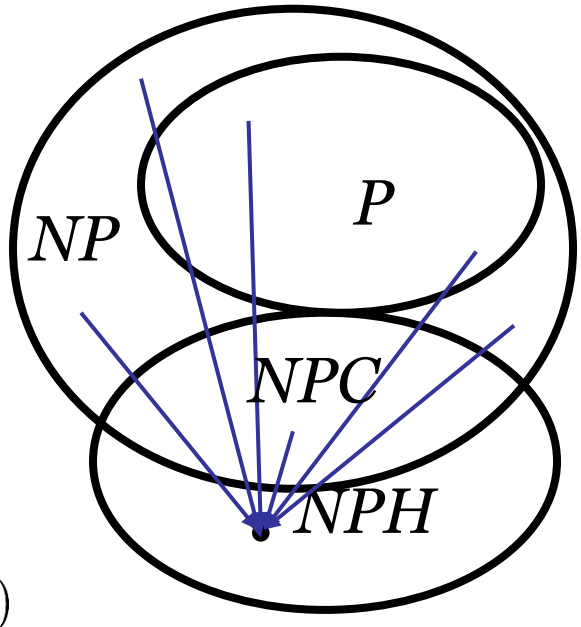


- **Fundamental question:** are some *NP* problems not in *P*?
  - $1,000,000 reward if you can answer
- *Inverse:* how would you prove *P* = *NP*?

- **Fact:** no one has *ever* proven an *NP* problem is not in *P*
  - Nor has anyone proven that it's impossible

NP-Hard

- Class of problems relevant to P vs. NP
- <u>Any problem</u> in NP can be reduced to an NP-Hard problem in polynomial-time
- If any NP-Hard problem is in *P*, all *NP* problems are in *P*
  - *P* = *NP*
  - "As hard as anything in *NP*"
- NP-Complete: *NP* problems that are NP-Hard
- Cook's Theorem (1971)
  - Boolean satisfiability is NP-Complete
- Boolean satisfiability (SAT)
  - **Input:** a Boolean expression
  - **Output:** can the expression be true?
  - **Example:** $(P \vee Q) \wedge (P \vee \neg Q) \wedge (\neg P \vee Q)$
    - Yes, if *P* and *Q* are both true
  - **Example:** $P \wedge \neg P$

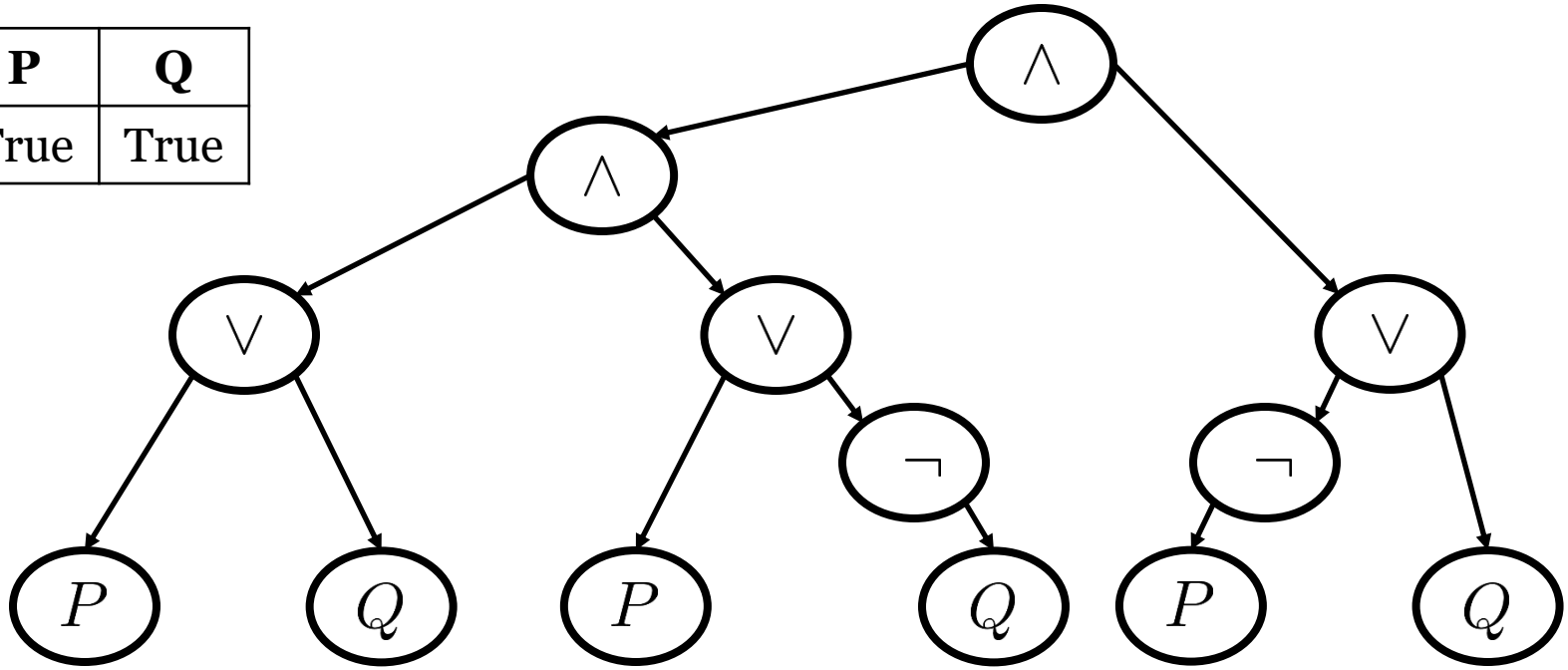


Cook's Theorem

- SAT is NP:
  - SAT can be verified in poly-time
  - Given expression and assignment of variables,
  - Construct expression tree
  - Evaluate expression tree

$$(P \vee Q) \wedge (P \vee \neg Q) \wedge (\neg P \vee Q)$$

P	Q
True	True



Cook's Theorem, part 2

- SAT is NP-Hard:
  - Much harder!

Main idea

- Let X be a decision problem in NP
- Algorithm to solve X:
  - Must be some Turing Machine T that verifies solution to X
    - Turing Machine: mathematical model for computers
  - Create a Boolean expression B that "runs" T and evaluates to return value
  - Evaluate SAT on B
  - If B is satisfiable, T must return true
    - Return true
  - Otherwise, return false
- **Complexity:** polynomial + SAT

What is the point of NPH and NPC?

- **Fact:** no one has *ever* produced a poly-time algorithm for an NP-Hard problem
  - If your problem is NP-Hard, don't try to solve it!

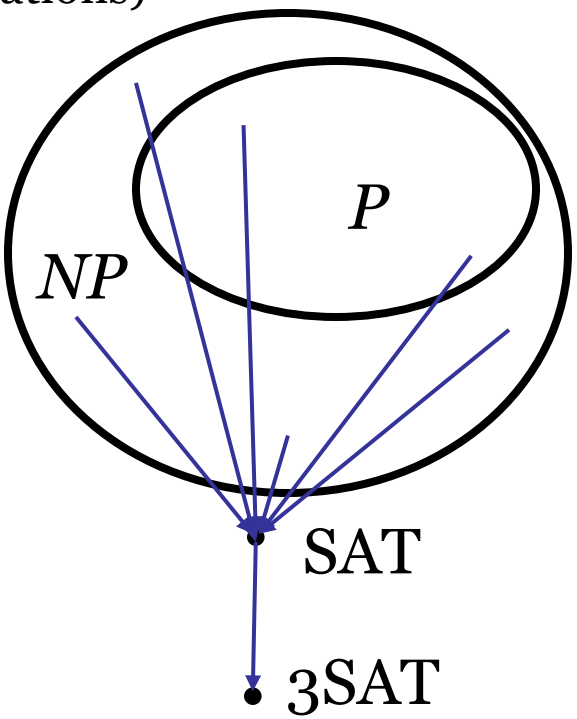
What can you do?

- Solve special case
  - E.g., longest path in acyclic graph
- Find heuristic algorithm
  - Tries to find a good answer
  - Not guaranteed to find optimum
  - E.g., greedy algorithm
- Find approximation algorithm
  - Tries to find a good answer
  - Solution guaranteed to be some fraction of optimal
  - E.g., within factor-2

I can't find an efficient algorithm, but neither can all of these smart people…

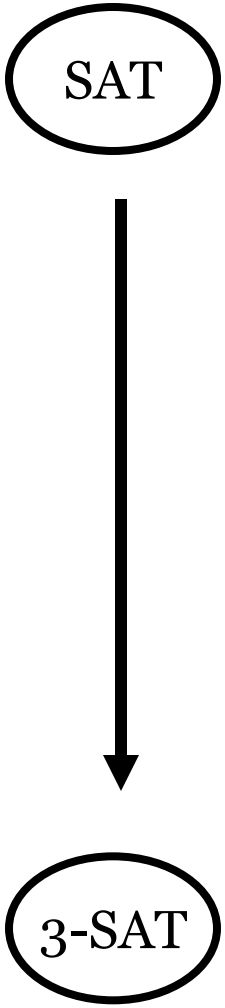
3-SAT

- Problem related to SAT
  - **Input:** Boolean expression in 3-CNF
    - Expression is "and" of *clauses*
    - Each clause is "or" of 3 *literals* (variables/negations)
  - **Example:** $(x \vee x \vee \neg y) \wedge (\neg x \vee y \vee y)$
  - **Output:** is expression satisfiable?
- **Goal:** prove 3-SAT is NP-Complete
  - Proving NP is easy
  - Same argument as SAT
- Prove NP-Completeness via reduction with SAT
  - Give poly-time reduction from SAT to 3SAT
  - Poly-time reduction from all NP to 3SAT



Input: *expr*: *any* boolean expression
Algorithm: SAT-to-3-SAT
$t =$ transform *expr* into 3-CNF
return 3SAT(t)

Problem reduction



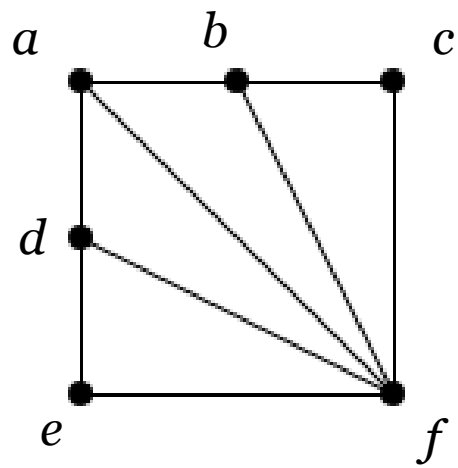
- Boolean Satisfiability
  - Known NP-Hard

- **Reduction**
  - Construct a 3-CNF expression *e'* with same satisfiability
    - Details in textbook (Section 17.3)
  - Return 3-SAT(*e'*)

- 3-CNF Boolean Satisfiability
  - Unknown (to prove)

Independent set

- **Input:** graph G
- **Output:** largest set of vertices with no edges between them

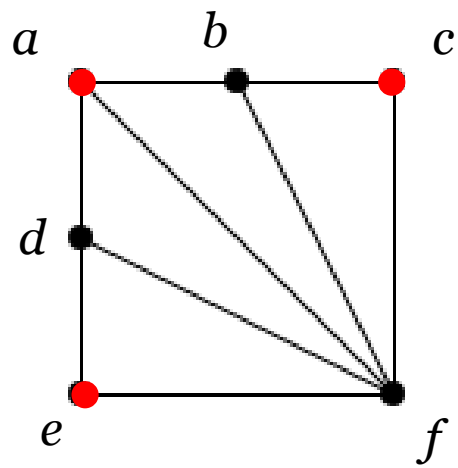


Decision problem:

- **Input:** graph G and integer k
- **Output:** does G have an IS of size k?

Independent set

- **Input:** graph G
- **Output:** largest set of vertices with no edges between them



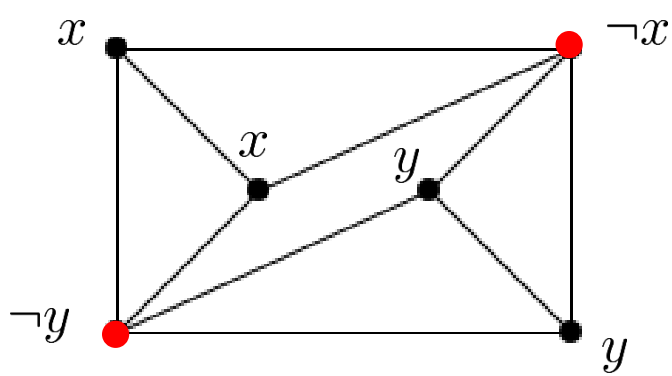
Decision problem:

- **Input:** graph G and integer k
- **Output:** does G have an IS of size k?
- IS is NP:
  - Given a potential IS
  - Mark all vertices in IS
  - Check that no vertex in IS has a marked neighbor

Problem reduction

3-SAT

- **3-CNF Boolean Satisfiability**
  - **Example:** $(x \vee x \vee \neg y) \wedge (\neg x \vee y \vee y)$
- **Reduction**
  - Construct graph G with vertex for each literal
    - Literal: variable or negated variable
  - Add edges between literals in same clause
  - Add edges between negations
  - Return IS(*G*, # *clauses*)



IS

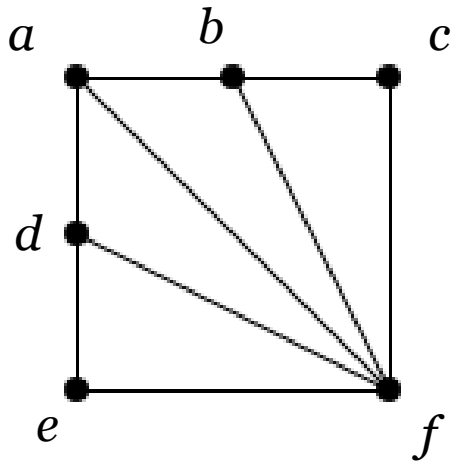
- **Independent set**

Problem reduction example

- **Complexity:** $O(n^2 + IS)$, where n is number of literals in expression
  - Poly-time
- Expression is satisfiable if IS(G, # clauses) returns true:
  - Assign false to all negated variables in IS, true to all positive variables
    - No variable is both true and false b/c of graph construction
  - Each clause must have one vertex in IS
  - This literal will be true
  - All clauses will be true
  - Expression will be satisfied
- If expression is satisfiable, IS(G, # clauses) returns true:
  - Each clause must contain at least one true literal
    - Choose one per clause
  - The corresponding vertices won't be adjacent in G
  - IS(G, # clauses) will be true

Clique

- **Input:** graph G
- **Output:** largest set of vertices fully connected by edges

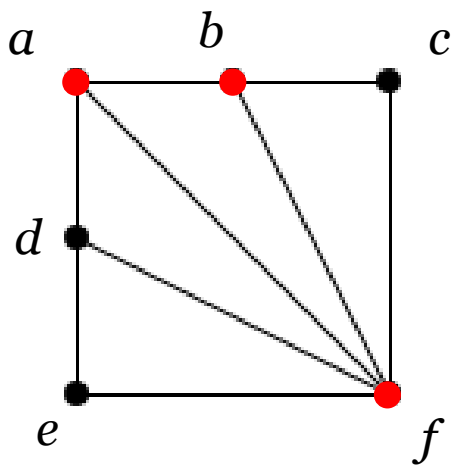


Decision problem:

- **Input:** graph G and integer k
- **Output:** does G have a clique of size k?

Clique

- **Input:** graph G
- **Output:** largest set of vertices fully connected by edges



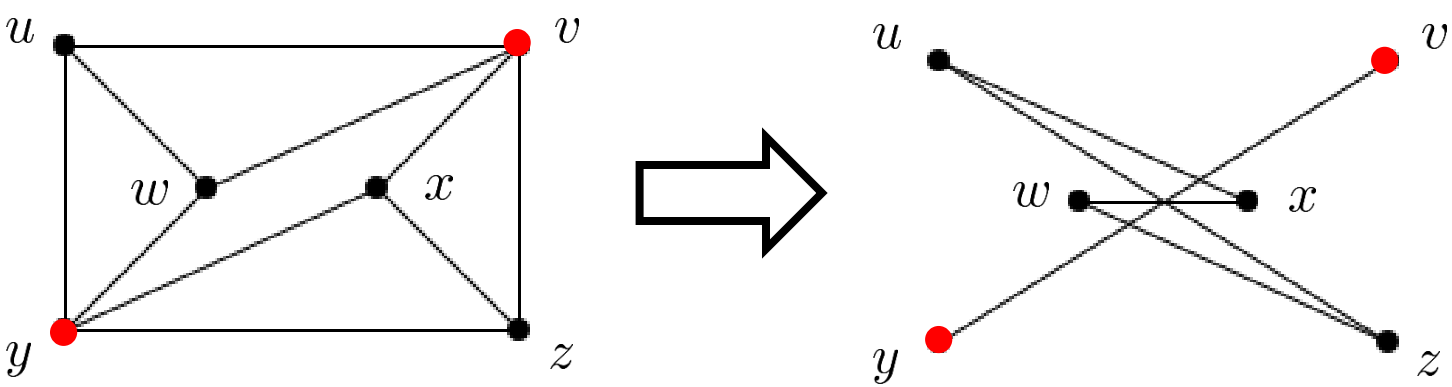
Decision problem:

- **Input:** graph G and integer k
- **Output:** does G have a clique of size k?
- Clique is NP:
  - Given a potential clique
  - Check every pair is adjacent

Problem reduction

IS

- **Independent set**
  - **Example:**



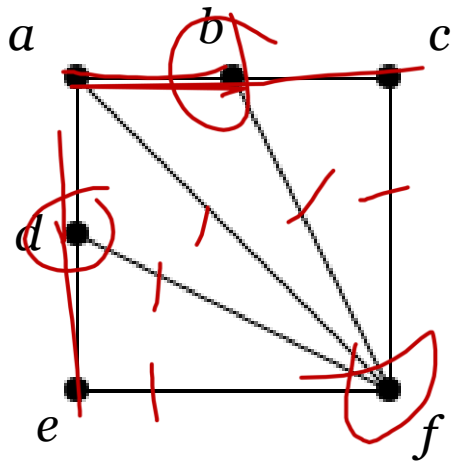
- **Reduction**
  - Construct *complement graph* G^c
    - Contains an edge iff original graph does not
  - Return Clique(G^c, k)

CLQ

- **Clique**

Vertex cover

- **Input:** graph G
- **Output:** smallest set of vertices adjacent to every edge

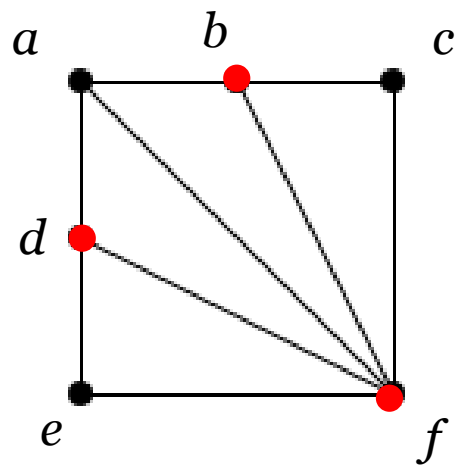


Decision problem:

- **Input:** graph G and integer k
- **Output:** does G have a VC of size k?

Vertex cover

- **Input:** graph G
- **Output:** smallest set of vertices adjacent to every edge



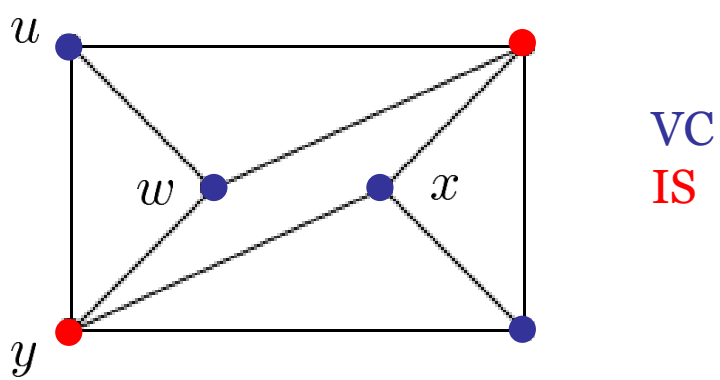
Decision problem:

- **Input:** graph G and integer k
- **Output:** does G have a VC of size k?
- VC is NP:
  - Given a proposed VC
  - Mark all incident edges
  - Check that all edges are marked

Problem reduction

IS

- **Independent set**
  - **Example:**



- **Reduction**
  - Return VC(G, n - k)
- **Justification**
  - VC covers all edges
  - Other vertices must be independent
  - And vice versa
- **Vertex cover**

VC

Coming up

- Final Exam review

- **Recommended readings:** Sections 17.1-17.3
- *Practice problems*: R-17.1, R-17.7, R-17.9, R-17.12, R-17.14, R-17.15